CSEN 240: MACHINE LEARNING

Project: Autonomous Multi-Agent Training for Table Tennis using Reinforcement Learning

- Inderpreet Kaur & Omkar Mulje

Table of Contents

Abstract....2
Introduction....3
Objective....3
Goal....3
Motivation and Benefits....4
Design and Architecture....4
1. System Overview....4
1.1 Single Agent Training....4
1.2 Multi-Agent Training....5
2. DDPG Architecture....5
3. Environment Design....6
3.1 State Space....6
3.2 Action Space....6
3.3 Reward System....7
3.4 Episode Termination....8
4. Training Setup....8
4.1 Single-Agent Training (DDPG)....8
4.2 Multi-Agent Training (MADDPG)....9
4.3 Training Analysis....11
Average Score per Episode....11
Individual Agent Performance....11
Success Rate per 100 Episodes....12
5. Hyperparameters....12
6. Results and Evaluation....13
6.1 Evaluation of Single-Agent DDPG Performance....13
6.2 Evaluation of Multi-Agent DDPG Performance....15
Challenges....16
Future Work....17

Abstract

In this project, we are building two virtual table tennis players that can teach themselves how to play using artificial intelligence. We use a method called Deep Deterministic Policy Gradient (DDPG). A type of reinforcement learning that helps agents learn by trying actions and improving over time. Each agent learns to control its paddle by observing the ball's position and speed, without any hardcoded rules. Over time, the agents get better at hitting the ball back and keeping a rally going. We also use smart memory sharing and stabilizing techniques to help them learn faster and more consistently.

By the end of the project, both agents should be able to play long rallies in a simulated table tennis environment. Our goal is to explore how well deep reinforcement learning works when two intelligent agents train together in a continuous and dynamic setting.

Introduction

In recent years, reinforcement learning (RL) has shown great potential in training intelligent agents to perform complex tasks through trial and error. Unlike traditional rule-based systems, RL allows agents to learn optimal behavior by interacting with their environment and receiving feedback. This project explores how reinforcement learning can be used to train two autonomous agents to play a simplified game of table tennis.

By using a method called Deep Deterministic Policy Gradient (DDPG), we aim to develop agents that can learn continuous paddle control and return the ball reliably. The focus is on creating a multi-agent system where each agent improves its performance over time without any hardcoded strategies. Through simulation, training, and visualization, this project demonstrates how AI agents can learn coordination and gameplay using only experience-based learning.

Objective

This project aims to develop a reinforcement learning system in which two autonomous agents learn to play table tennis against each other in a simulated environment. The core objective is to apply the Deep Deterministic Policy Gradient (DDPG) algorithm to train each agent to:

- Control its paddle with precision in a continuous action space
- Interpret state inputs such as ball position, velocity, and paddle location
- Learn through interaction and feedback without any hardcoded rules
- Achieve longer rallies by minimizing drop rate and improving coordination

The ultimate goal is to demonstrate how deep reinforcement learning techniques can enable agents to learn complex, real-time decision-making in a multi-agent setting.

Goal

The goal of this project is to develop a simulation where two autonomous agents can learn to play table tennis against each other using deep reinforcement learning techniques. By applying the DDPG algorithm, the project aims to demonstrate how agents can learn coordination and control in a continuous, multi-agent environment without relying on predefined rules or strategies.

Motivation and Benefits

In real-world scenarios, many tasks involve more than one agent working together or competing, like in sports, traffic systems, or robotics. Training intelligent agents to interact and adapt in such dynamic environments is a key challenge in artificial intelligence.

This project is motivated by the idea of teaching two AI agents to learn a skill that is playing table tennis without any manual instructions. It explores how agents can develop coordination, timing, and strategy purely through reinforcement learning.

The benefits of this project are:

- Gaining practical experience in implementing deep reinforcement learning (DRL) algorithms.
- Understanding how agents learn from feedback and improve through trial and error.
- Exploring multi-agent learning, where agents must consider not just their actions but also the behavior of others.
- Building a foundation for real-world applications such as collaborative robotics, autonomous vehicles, and competitive simulations

Design and Architecture

1. System Overview

In our project, we first train a single agent using the Deep Deterministic Policy Gradient (DDPG) algorithm in a custom environment. After verifying the performance, we move to a multi-agent setup using the Multi-Agent Deep Deterministic Policy Gradient (MA-DDPG) algorithm.

1.1 Single Agent Training

The single agent is trained in a simplified table tennis environment built using Python in a Jupyter notebook. The environment simulates a paddle trying to return a ball. We use the DDPG algorithm, which combines actor-critic methods and target networks to handle continuous action spaces.

- The agent observes the environment (ball position, velocity, paddle position)
- It decides how to move the paddle
- It receives a reward for successfully hitting the ball
- Over time, the agent learns to hit the ball more consistently

1.2 Multi-Agent Training

Once the single agent performs well, we extend the system to a multi-agent version using the Tennis environment provided by Unity and Udacity.
Here, two agents (paddles) play against each other. Each has its own Actor network, but the Critic can see actions and states from both agents to improve cooperative learning.

We adapt the MA-DDPG algorithm, which builds on DDPG but adds support for inter-agent coordination.

2. DDPG Architecture

Deep Deterministic Policy Gradient (DDPG) is a type of reinforcement learning algorithm that works well when an agent has to choose continuous actions — like moving a paddle up or down smoothly instead of picking from a fixed set of choices.
It belongs to a group of methods called Actor-Critic algorithms. In DDPG:

- The Actor is a neural network that decides what action to take in a given situation (or state).
- The Critic is another neural network that tells the agent how good that action is.

DDPG also uses:

- A Replay Buffer – this stores the agent's past experiences so it can learn from them later, by picking random samples.
- Target Networks – these are slow-changing copies of the actor and critic networks that help make learning more stable.

In our project, DDPG is used to train an agent to play table tennis. The agent sees the position and speed of the ball, and the position of the paddle. Then, it decides how to move the paddle up or down to try and hit the ball. The agent gets a positive reward when it hits the ball, and a negative reward when it misses. Over many episodes, it learns how to improve its timing and accuracy to hit the ball more often.

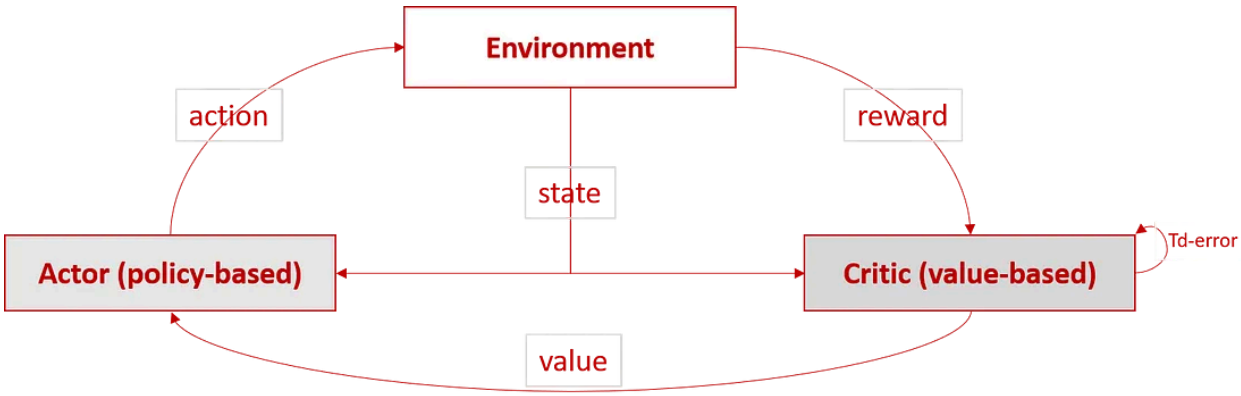

Fig 1: DDPG Architecture [2]

3. Environment Design

We built a simple table tennis environment using Python inside a Jupyter notebook. This environment simulates the motion of a ball and a single paddle that the agent controls. The environment was implemented using NumPy and custom logic without relying on any external simulator.

3.1 State Space

Each observation (state) is a vector of 5 values:

Variable	Description
ball_x	Horizontal position of the ball
ball_y	Vertical position of the ball
ball_vx	Horizontal velocity of the ball
ball_vy	Vertical velocity of the ball
paddle_y	Current vertical position of the paddle

Table 1: Description of the agent's observation (state) vector

These values help the agent understand where the ball is and how to move the paddle.

3.2 Action Space

In our table tennis environment, the action space defines what the agent can do at each step — in other words, how it can move the paddle. We are using a continuous action space, meaning

the agent can choose any real number in a certain range, instead of picking from fixed actions like "up", "down", or "stay". The action is a single float value (e.g., -0.5, 0, +1.0) representing vertical movement of the paddle.

3.3 Reward System

The reward is designed to encourage the agent to hit the ball accurately:

- If the ball reaches the paddle, the environment checks how close the paddle's vertical position is to the ball's position.
- If the paddle misses the ball, the agent receives a reward of -1 and the episode ends.
- For all other steps, the reward is 0.

The reward is computed in the step() function of the TableTennisEnv class as shown below:

```
def step(self, action):
    self.paddle_y += action[0] * self.paddle_speed * self.dt
    self.paddle_y = np.clip(self.paddle_y, 0, 1)
    self.ball_pos += self.ball_vel * self.dt

    if self.ball_pos[1] <= 0 or self.ball_pos[1] >= self.height:
        self.ball_vel[1] *= -1

    reward = 0.0
    if self.ball_pos[0] <= 0.05:
        dist = abs(self.ball_pos[1] - self.paddle_y)
        if dist < 0.1:
            self.ball_vel[0] *= -1
            reward = max(0, 1.0 - dist * 3)

        else:
            reward = -1.0
            self.done = True

    if self.ball_pos[0] < 0:
        self.done = True

    return self._get_state(), reward, self.done, {}

def _get_state(self):
    return np.concatenate([self.ball_pos, self.ball_vel, [self.paddle_y]])
```

Fig 2: Reward function for single agent

3.4 Episode Termination

An episode represents one complete sequence of interaction between the agent and the environment — from start (reset) to end (done). In our table tennis setup, an episode can end in two ways:

1. Paddle Misses the Ball

- The episode ends immediately when the ball reaches the paddle zone and the paddle fails to hit it.
- This is determined when:
    - The ball's horizontal position (ball_x) is less than or equal to 0.05 (paddle zone), and
    - The vertical distance between the ball and paddle is greater than 0.1.

- In this case, the agent receives a reward of -1, and done = True.

2. Maximum Steps Reached

- If the paddle keeps returning the ball successfully, the environment still limits the interaction to a fixed number of time steps (like 200).
- If this step limit is reached, the episode ends naturally with done = True, even if the game is still going.

4. Training Setup

4.1 Single-Agent Training (DDPG)

For the single-agent implementation, we designed a basic table tennis environment using Python and NumPy. Training was performed in a Jupyter Notebook using the PyTorch framework.

- No external simulation engine was required.
- The environment was lightweight and runs efficiently on standard CPU-based systems.
- Training can be done on platforms like Google Colab or any regular laptop.
- The agent was trained over multiple episodes to learn how to control the paddle and hit the ball back.

This setup is ideal for quick experimentation and understanding the fundamentals of reinforcement learning in a simple 2D space.

4.2 Multi-Agent Training (MADDPG)

Training Setup

- Environment: Unity-based table tennis simulation with two agents controlling paddles on either side of the table.
- Episodes: 2000 total training episodes were used based on early testing which showed performance plateauing after 1500–1800 episodes.

- Target Metric: The environment was considered "solved" once the average score per episode exceeded 0.5 over the last 100 episodes (the official Unity baseline).

In the first stage, we trained both agents using default hyperparameters. The environment was marked as solved after 717 episodes, reaching an average score of 0.53. However, training progress was slow and noisy in early episodes.

```
Episode 50      Avg Score: 0.01 avg max score over the last 10 episodes: 0.00
Episode 100     Avg Score: 0.02 avg max score over the last 10 episodes: 0.07
Episode 150     Avg Score: 0.01 avg max score over the last 10 episodes: 0.00
Episode 200     Avg Score: 0.06 avg max score over the last 10 episodes: 0.06
Episode 250     Avg Score: 0.04 avg max score over the last 10 episodes: 0.02
Episode 300     Avg Score: 0.05 avg max score over the last 10 episodes: 0.10
Episode 350     Avg Score: 0.07 avg max score over the last 10 episodes: 0.08
Episode 400     Avg Score: 0.06 avg max score over the last 10 episodes: 0.04
Episode 450     Avg Score: 0.08 avg max score over the last 10 episodes: 0.07
Episode 500     Avg Score: 0.08 avg max score over the last 10 episodes: 0.06
Episode 550     Avg Score: 0.09 avg max score over the last 10 episodes: 0.10
Episode 600     Avg Score: 0.10 avg max score over the last 10 episodes: 0.13
Episode 650     Avg Score: 0.10 avg max score over the last 10 episodes: 0.10
Episode 700     Avg Score: 0.10 avg max score over the last 10 episodes: 0.10
Episode 750     Avg Score: 0.13 avg max score over the last 10 episodes: 0.16
Episode 800     Avg Score: 0.21 avg max score over the last 10 episodes: 0.33
Episode 817     max score: 2.60 avg max score over the last 10 episodes: 1.35
Environment solved in 717 episodes!     Average Score: 0.53
```

Fig 3: Stage 1: Baseline Configuration

We conducted a second experiment by applying several optimizations:

- Increased the number of learning steps per episode.
- Increased initial exploration noise to encourage broader state exploration, followed by quick decay.
- Reduced the batch size to allow for more frequent updates.
- Adjusted learning rates for both actor and critic networks to promote smoother convergence.
- As a result, the environment was solved in just 223 episodes, with an average score of 0.50, significantly improving sample efficiency and convergence speed.

```
Episode 50      Avg Score: 0.01 avg max score over the last 10 episodes: 0.00
Episode 100     Avg Score: 0.02 avg max score over the last 10 episodes: 0.04
Episode 150     Avg Score: 0.06 avg max score over the last 10 episodes: 0.08
Episode 200     Avg Score: 0.09 avg max score over the last 10 episodes: 0.09
Episode 250     Avg Score: 0.10 avg max score over the last 10 episodes: 0.10
Episode 300     Avg Score: 0.14 avg max score over the last 10 episodes: 0.22
Episode 323     max score: 0.30 avg max score over the last 10 episodes: 0.48
Environment solved in 223 episodes!     Average Score: 0.50
```

Fig 4: Stage 2: Optimized Configuration

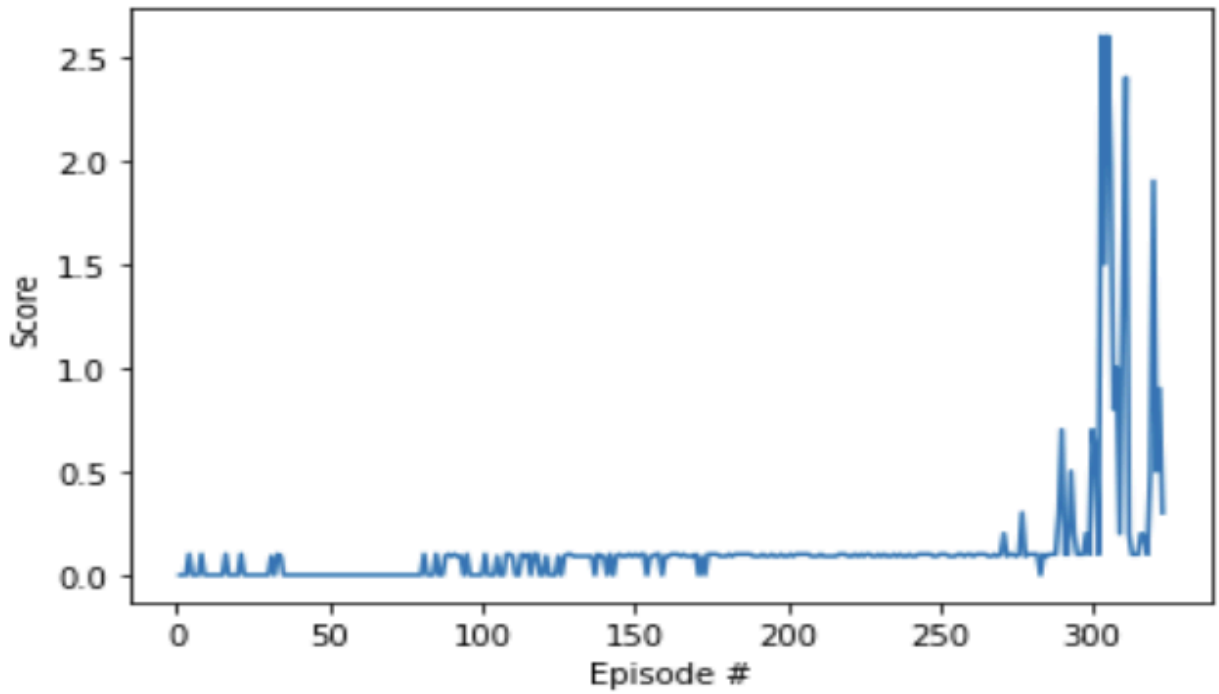

Fig 5: Training score per episode

Following the success of Stage 1, a more extended training run was conducted to assess long-term performance and stability. The agents were trained using the improved MADDPG configuration over approximately 1400 episodes in a multi-agent table tennis environment. The objective was to evaluate consistency, coordination, and the ability to reach and sustain high-performance thresholds.

4.3 Training Analysis

```
=== TRAINING SUMMARY ===
Episodes: 1413
Best Score: 1.680
Final Success Rate: 60.0%
Agent 1 Mean: 0.956
Agent 2 Mean: 0.956
```

Fig 6: Training Summary with 60% success rate

Average Score per Episode

The training progression revealed a significant upward trend in average scores beginning around episode 300. From episodes 600 to 1200, the agents consistently surpassed the solved threshold of 0.5, which is the Unity benchmark for task success. This reflects meaningful policy learning and improved gameplay dynamics over time.

Fig 7: Average Score per Episode

Individual Agent Performance

Both agents demonstrated similar performance throughout training. The learning curves overlapped closely, with the highest performance observed between episodes 800 and 1000. This behavior confirms synchronized policy improvement and agent cooperation.

Fig 8: Individual Agent Performance

Success Rate per 100 Episodes

The success rate, defined as the percentage of episodes where the average score exceeded 0.5, reached 60% by the end of training. Improvement was gradual and steady, with success rate plateauing in the later training stages, indicating stability in agent behavior.

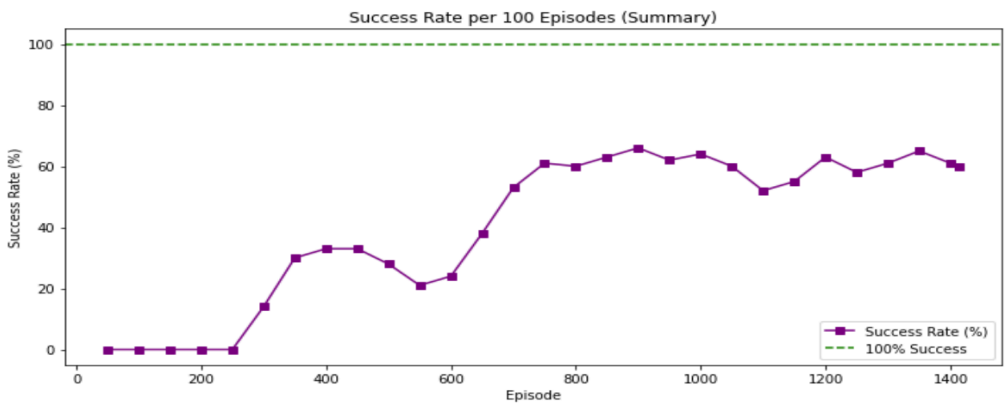

Fig 9: Success Rate per 100 Episodes

5. Hyperparameters

The table shows the key hyperparameters used during training. The values were chosen based on common DDPG implementations and were adjusted slightly during experimentation to achieve better stability.

Parameter	Value
Number of Episodes	2000
Max Steps per Episode	1000
Actor Learning Rate	1.00E-04
Critic Learning Rate	1.00E-03
Discount Factor (Gamma)	0.99
Soft Update Factor (Tau)	0.001
Replay Buffer Size	100,000
Batch Size	64
Action Noise Std. Dev	0.15 → decayed

Table 2: Key hyperparameters for multi agent

6. Results and Evaluation

6.1 Evaluation of Single-Agent DDPG Performance

To evaluate the learning progress of our DDPG-based single agent, we trained the agent over 100, 200, 500, 1000 episodes and tracked the total reward per episode. The reward represents how well the paddle was able to return the ball over time.

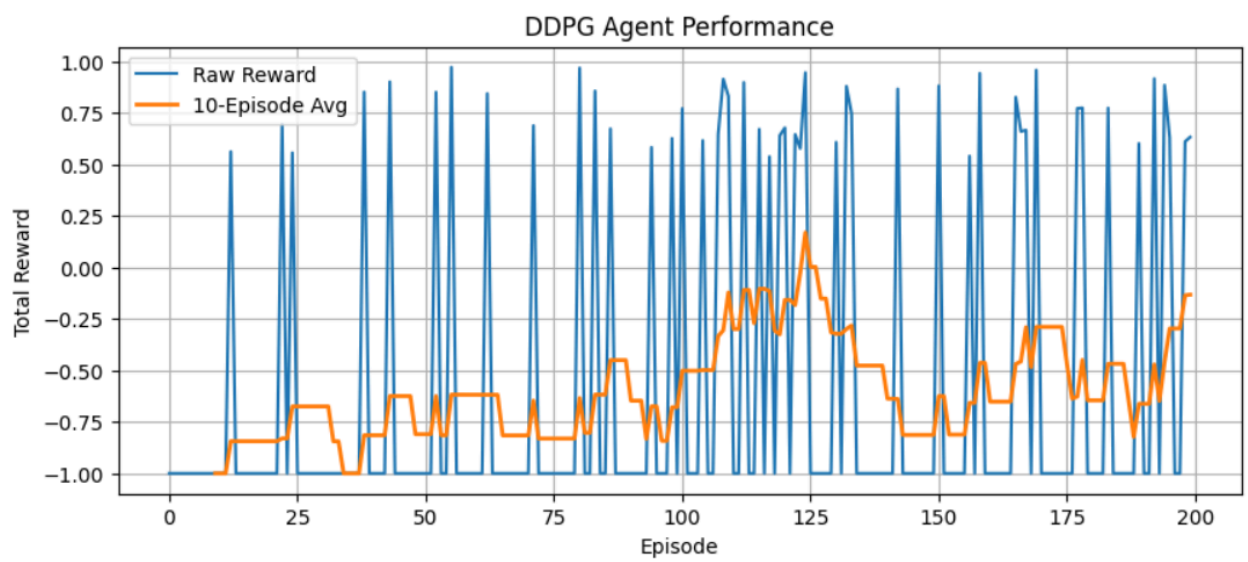

Fig 10: Single Agent Performance over 200 episodes

Episodes- 200. The orange line stays close to -1.0 for the first 100 episodes → still struggling to hit early on. Around episode 120–150, the smoothed curve starts to rise — this means the agent is having more consistent success during that range.

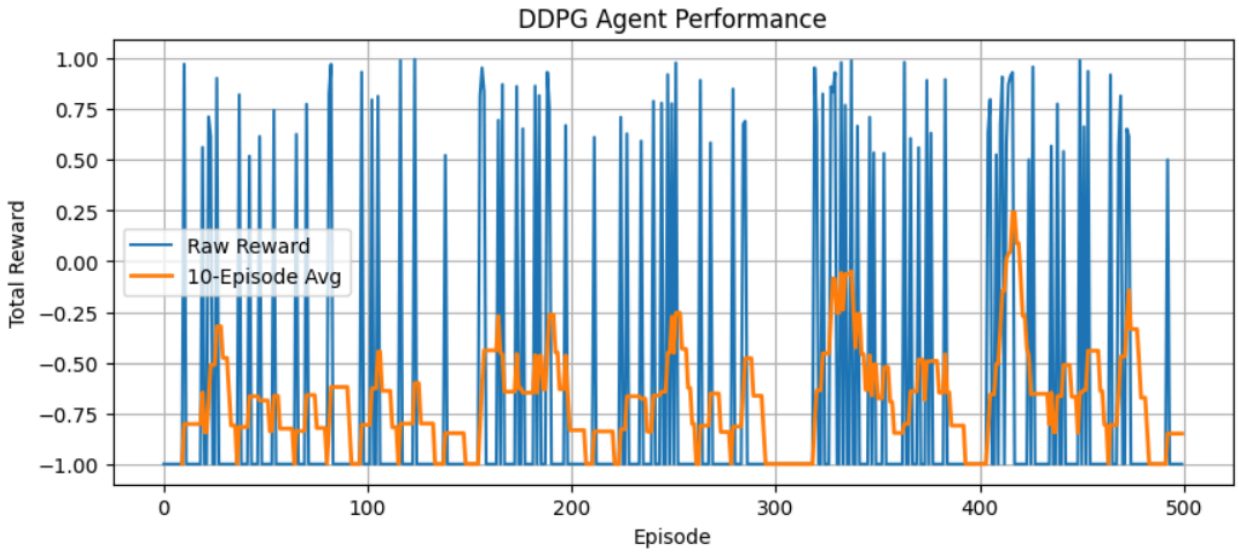

Fig 11: Single Agent Performance over 500 episodes

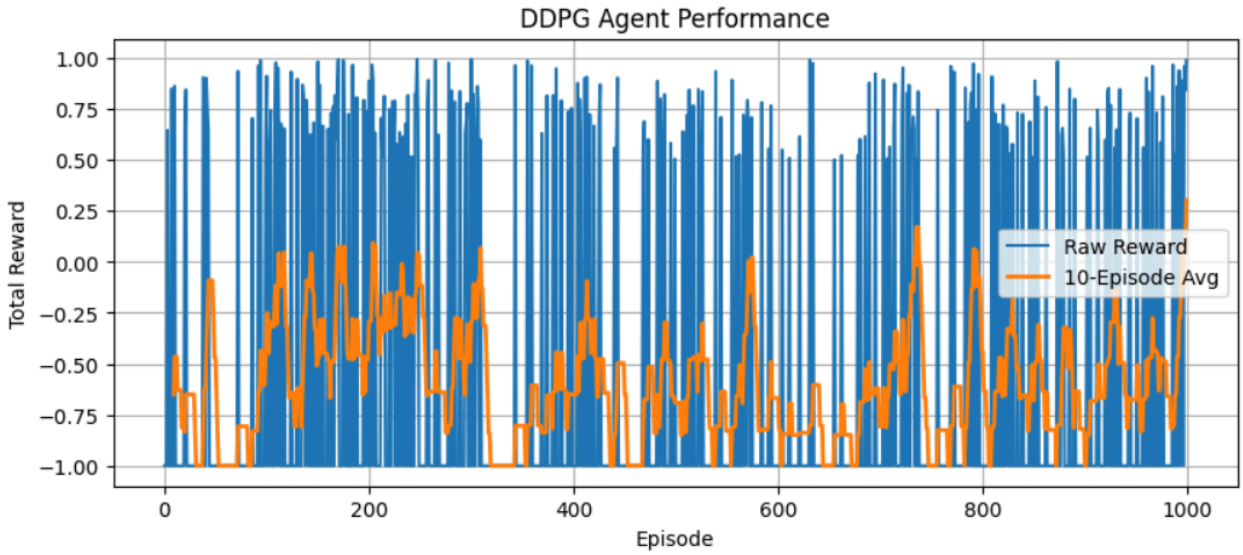

Fig 12: Single Agent Performance over 1000 episodes

For 1000 episodes the DDPG agent learned to return the ball with increasing accuracy over time using continuous action control and shaped rewards.

Observations from Reward Graphs of Single Agent

- In the early episodes, the reward was mostly negative, indicating that the agent frequently missed the ball.

- As training progressed, the agent began learning to position the paddle better, resulting in more frequent hits and a gradual increase in reward.

- After around 500 episodes, the curve started to show stable upward trends, reflecting more consistent behavior.

- However, due to the simplicity of the environment and limited state-action space, the learning curve occasionally plateaued or fluctuated slightly.

Although the agent does not achieve perfect performance, it demonstrates the ability to return the ball occasionally. The reward plot shows learning spikes, indicating that the agent is beginning to understand paddle control using deep reinforcement learning. Further improvements can be made by tuning hyperparameters or increasing training duration

6.2 Evaluation of Multi-Agent DDPG Performance

The trained agents were evaluated over 10 independent test episodes in order to assess generalization, stability, and consistency of learned behaviors. The evaluation was performed without further learning or exploration noise, allowing for a clear observation of performance under a fixed policy.

```
============================================================
Testing trained agent for 10 episodes
============================================================

Episode  1: Max Score =  0.100, Agent Scores = [-0.010,  0.100], Length =   31 steps
Episode  2: Max Score =  2.600, Agent Scores = [ 2.490,  2.600], Ler th =  983 steps
Episode  3: Max Score =  0.000, Agent Scores = [ 0.000, -0.010], Length =   13 steps
Episode  4: Max Score =  2.600, Agent Scores = [ 2.600,  2.600], Length = 1000 steps (timeout)
Episode  5: Max Score =  2.700, Agent Scores = [ 2.600,  2.700], Length = 1000 steps (timeout)
Episode  6: Max Score =  2.600, Agent Scores = [ 2.600,  2.600], Length = 1000 steps (timeout)
Episode  7: Max Score =  0.200, Agent Scores = [ 0.200,  0.090], Length =   54 steps
Episode  8: Max Score =  0.090, Agent Scores = [ 0.000,  0.090], Length =   31 steps
Episode  9: Max Score =  0.100, Agent Scores = [ 0.100, -0.010], Length =   31 steps
Episode 10: Max Score =  0.100, Agent Scores = [-0.010,  0.100], Length =   32 steps
```

Fig 13: Episode summary of agent performance over 10 test episodes

As shown in Fig 13, 4 out of 10 test episodes were considered successful, where the agents reached a max score above the target threshold of 0.5. These successful episodes had significantly longer durations, including multiple instances where the agents sustained gameplay

for the full 1000 steps (episode timeout). In contrast, unsuccessful episodes tended to terminate early due to missed returns, often within 30–50 steps.

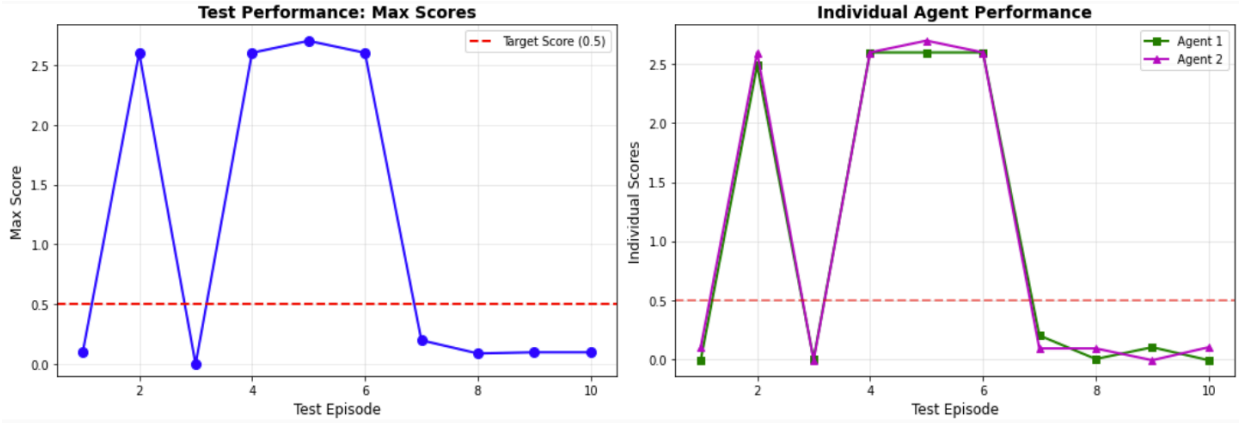

Fig 14: Test performance across episodes showing max scores (left) and individual agent scores (right) against the target threshold.

In the Max Scores graph, episodes 2–6 show high performance above the target score (0.5), while the rest fall below, indicating inconsistent success across test runs.
In the Individual Agent Performance graph, both agents show closely aligned scores in successful episodes, confirming coordinated learning. Lower episodes show a steep drop for both agents, highlighting instability in generalization.

Challenges

1. Training Time: Training multi-agent reinforcement learning systems proved to be computationally intensive. A large number of episodes were required to achieve meaningful learning, particularly due to the presence of Ornstein-Uhlenbeck (OU) noise and the sensitivity of learning to step size. These factors contributed to longer convergence times and necessitated significant training resources.

2. Achieving Success Criteria, Efficiency, and Stability: Consistently reaching the defined success threshold (90% success rate) across episodes remained a key challenge. While the agents demonstrated potential through high-scoring episodes, their performance was not stable throughout testing. Variance in reward signals, exploration noise, and occasional early episode terminations led to fluctuations in agent behavior and reduced overall reliability.

Future Work

1. Future improvements could include the use of adaptive learning mechanisms such as curriculum learning or dynamic difficulty adjustment. These methods would allow agents to start with simpler tasks and gradually face more complex gameplay situations, potentially accelerating convergence and improving performance consistency.

2. The current environment is simplified for training efficiency. Future work may involve the creation of a more physics-accurate custom environment by introducing elements such as spin, paddle angle variation, and air resistance. These additions would help the agents learn in conditions that more closely resemble real-world dynamics.

3. To enhance learning stability and exploration efficiency, alternative reinforcement learning algorithms could be explored. Candidates include TD3 (Twin Delayed DDPG), SAC (Soft Actor-Critic), and multi-agent variants such as MA-PPO. These algorithms are known to provide smoother convergence and better performance in complex continuous control settings.

References

[1] Wang, Sen, Daoyuan Jia, and Xinshu Learning in Robotic Table Tennis," *2021 IEEE International Conference on Robotics and Automation (ICRA)*, Xi'an, China, 2021, pp. 4171-4178, doi: 10.1109/ICRA48506.2021.9560764.

[2] Project 3: Training two deep-RL agents to play Tennis | by Jasdeep Sidhu | Medium

[3] L. Yang, H. Zhang, X. Zhu and X. Sheng, "Ball Motion Control in the Table Tennis Robot System Using Time-Series Deep Reinforcement Learning," in *IEEE Access*, vol. 9, pp. 99816-99827, 2021, doi: 10.1109/ACCESS.2021.3093340

[4] Y. Zhu, Y. Zhao, L. Jin, J. Wu and R. Xiong, "Towards High Level Skill Learning: Learn to Return Table Tennis Ball Using Monte-Carlo Based Policy Gradient Method," *2018 IEEE International Conference on Real-time Computing and Robotics (RCAR)*, Kandima, Maldives, 2018, pp. 34-41, doi: 10.1109/RCAR.2018.8621776.